Minimal SWE Agent 学习问题及解答（续篇）

基于 2026-10-03 之后的项目对话整理 · 2026-10-24

用途：接续上一份《10.03_Minimal_SWE_Agent_问题及解答》，整理后续开发中提出的新问题。答案保持简短，重点记录工具测试、Agent Loop、API
排查、首次真实修复与局部编辑工具。已在上一份文档中解释过的基础概念不再重复展开。

一、TerminalTool 与命令执行

Q1. subprocess.run("python -m pytest", shell=True) 为什么不安全？

答：shell=True 会让命令经过系统 Shell
解释；如果命令字符串混入不可信内容，可能被拼接成额外命令。优先使用参数列表并关闭 Shell，例如 subprocess.run([sys.executable, "-m", "pytest"], shell=False)。

Q2. TerminalTool 为什么需要单独写测试？

答：它会真正启动外部进程，需验证命令执行、输出捕获、返回码、超时和异常处理。测试可使用受控命令或临时目录，避免影响真实项目。

Q3. 为什么第一次接入真实 LLM 时要限制终端命令？

答：模型会自主选择工具。第一版只允许明确的测试命令，可以减少误执行其他命令的风险；但命令白名单不等于操作系统沙箱，pytest 仍会运行测试代码。

Q4. 测试中的 helper.py 需要提前手动创建吗？

答：不一定。Path 对象只表示路径；mkdir() 创建目录，write_text()
在父目录存在时可创建或覆盖文件。测试里动态创建的 helper.py 只是临时测试数据，不是 Agent 源码的一部分。

Q5. pytest 的 passed 数量为什么不等于 assert 数量？

答：pytest 通常按测试函数或测试用例计数，而不是按 assert 语句计数。一个测试函数里可以有多个 assert，但通常仍只算一个 passed。

二、Parser、PromptBuilder 与 Agent Loop

Q1. Parser 测试主要验证什么？

答：验证模型返回的文本能否被解析成 Action，包括普通 JSON、Markdown 的 JSON
代码块，以及格式错误时的处理。

Q2. PromptBuilder 的作用是什么？

答：把任务说明、可用工具及其描述、执行历史和当前状态整理成 Prompt。它负责组织输入，不负责调用 LLM，也不直接执行工具。

Q3. Agent Loop 每轮主要做什么？

答：构建 Prompt → 调用 LLM → Parser 解析 Action → 判断是否 finish → Registry 查找工具 → 执行工具得到 Observation → 保存历史并增加步数 → 进入下一轮。

Q4. max_steps=2 时，两个预设 response 都会执行吗？

答：如果两个 response 都是普通工具 Action，且每轮执行成功进入下一轮，那么两轮都会被消费；step_count 到 2 后循环停止。达到步数上限时调用 finish 不一定会增加一条 Action 历史。

Q5. Agent 的 finished=True 是否一定代表修复成功？

答：不一定。它只表示 Agent 已结束；可能是正常 finish，也可能是达到最大步数。当前版本主要依靠模型判断是否结束，还没有独立的程序级验收机制强制要求测试通过。

Q6. 为什么 Agent 运行时终端长时间没有输出？

答：如果 main.py 只在 agent.run() 返回后打印历史，执行期间就看不到进度。可能是在等待 LLM API，也可能在执行工具。可在请求前后加 print(..., flush=True)，并记录当前步骤与工具名称。

三、测试目录与真实修复任务

Q1. 为什么 test_calculator.py 放在 workspace，而不是 tests？

答：tests/ 用来测试 SWE Agent 框架自身；workspace/ 放置 Agent 要处理的目标项目。两者分开可以避免故意失败的目标测试污染框架测试结果。

Q2. 为什么要先让 calculator.py 的测试失败，再运行 Agent？

答：先确认 Bug 确实存在，才能验证 Agent 是否真正修复了问题。修复后再次运行目标测试，才能形成“失败 → 修改 → 通过”的完整证据链。

Q3. 第一次真实 SWE Agent 运行了哪几步？

答：实际轨迹为：read_file(calculator.py) → read_file(test_calculator.py) → write_file(calculator.py) → terminal 运行 pytest → finish。测试结果为 1 passed，证明多个模块已能协同完成简单修复。

Q4. 为什么框架测试和 workspace 测试要分开运行？

答：uv run python -m pytest tests 只测试 Agent 框架；uv run python -m pytest workspace 测试目标项目。两者的 passed 数量应分别记录，不能混为同一组测试。

四、API 超时排查与模型切换

Q1. TcpTestSucceeded=True 能证明 API 正常吗？

答：不能。它只表示 TCP 端口能够建立连接，不代表 HTTPS 请求、鉴权、模型路由或推理服务正常。

Q2. curl 请求 /v1/models 返回 401，说明什么？

答：如果请求没有携带 API Key，401 和 Invalid token 是预期的鉴权拒绝；这不能证明密钥失效。使用已配置密钥的客户端成功查询模型列表，才说明该查询接口的鉴权通过。

Q3. deepseek-v3 出现在模型列表中，为什么调用仍然超时？

答：模型列表只说明平台列出了这个 ID，不保证其推理路由当前可用。调用超时不能单独证明模型已下线；还需结合其他模型的对照请求判断。

Q4. 如何区分网络问题和模型服务问题？

答：先检查 DNS/TCP/TLS，再用带有效密钥的 /v1/models 查询接口，最后对聊天接口做最小请求。不同层级的结果能逐步缩小问题范围，但单一检查不能证明整个服务都正常。

Q5. 为什么最后改用 deepseek-flash？

答：deepseek-v3 多次在约 20 秒内超时；同一配置改为 deepseek-flash 后返回“API连接成功”，耗时约 5.09 秒。因此项目暂时使用 deepseek-flash，避免 Agent 多轮调用被慢请求拖住。

Q6. 如果日志仍显示 DeepSeek-V3，但 .env 已改为 deepseek-flash，怎么办？

答：检查日志文本是否写死在 print() 中。应动态打印实际读取的 LLM_MODEL，避免日志与真正调用的模型不一致。

五、ReplaceTextTool：局部代码修改

Q1. 为什么在 WriteFileTool 之外增加 ReplaceTextTool？

答：WriteFileTool 会完整覆盖文件；ReplaceTextTool 只替换指定文本片段，能减少 LLM 重写整份文件时误删其他代码的风险。

Q2. ReplaceTextTool 的第一版安全规则是什么？

答：只能访问 workspace 内文件；old_text 不能为空；旧文本不存在时拒绝修改；旧文本出现多次时也拒绝修改，要求提供更精确的匹配内容。

Q3. 为什么替换前要统计 old_text 出现次数？

答：用 str.count() 判断匹配次数。只有恰好出现一次时才替换，避免同一片段在多个函数中重复出现而导致误修改。

Q4. ReplaceTextTool 测试覆盖了哪些情况？

答：正常替换且保留其他内容、找不到旧文本、旧文本重复、参数错误、文件不存在、路径越界。新增 6 个测试后，框架测试预期由 38 passed 增至 44 passed。

Q5. 将 ReplaceTextTool 接入 Agent 需要改哪些地方？

答：在 main.py 导入并注册工具，并在任务说明中加入 replace_text。ToolRegistry 保存工具后，PromptBuilder 可把它纳入可用工具说明，Agent Loop 核心流程不需要重写。

六、当前进度与快速记忆

- 已完成：LLM 调用、AgentState、BaseTool、ToolRegistry、文件读写与搜索、TerminalTool、ActionParser、PromptBuilder、Agent Loop。
- 已验证：第一次真实 Agent 成功修复 calculator.py；ReplaceTextTool 已完成单元测试并接入 Agent。
- 测试基线：Agent 框架测试 44 passed；workspace 演示项目修复后 1 passed。两组测试独立运行。
- 核心链路：任务 → Prompt → LLM → ActionParser → ToolRegistry → Tool → Observation → AgentState → 下一轮。
- 重要限制：API 异常目前可能直接中断；达到 max_steps 也会结束；当前没有独立的 “ 测试必须通过才允许 finish ” 验收机制。
- 代码输出习惯：测试代码尽量放在一个完整代码块中，逐行加入必要注释；给出测试代码时同时说明预计 passed 数量。

本续篇记录截至 2026-10-24 的学习进度。模型名称、测试数量以项目当前实际代码和运行输出为准。